

Global Sales Data Warehouse for Adaptive Analytical Query Optimization

Bhavya Patel
University of Windsor
Windsor, Canada
patel61d@uwindsor.ca

Yash Prajapati
University of Windsor
Windsor, Canada
prajap6k@uwindsor.ca

Harsh Patel
University of Windsor
Windsor, Canada
patel41d@uwindsor.ca

Venkata Charan Sai Doniparthi
University of Windsor
Windsor, Canada
donipar@uwindsor.ca

Abstract—This midterm status report documents progress on an adaptive, workload-aware physical-design engine for a PostgreSQL 16 star-schema data warehouse. The project compares a static baseline physical design against a closed-loop adaptive engine that is intended to monitor `pg_stat_statements`, detect hot access paths, and issue guarded runtime index changes, across a 3×4 matrix of query-workload types and concurrent ETL ingestion rates. As of this report, Phase 1 (environment, star schema, static baseline indexes, experiment-logging schema, and a skewed synthetic data loader) is complete and verified end-to-end against a live database. Phases 2 through 4, comprising the workload generator, ETL injector, adaptive engine, and the full comparative experiment, are designed in detail below but not yet present in the project repository, and are reported honestly as remaining work. This report presents the task breakdown, dependency structure, version-control evidence, the target system architecture, and a live-demonstrable prototype of the data layer, together with the plan for the remaining three implementation phases.

Index Terms—adaptive indexing, data warehouse, query optimization, PostgreSQL, OLAP, star schema, workload-aware tuning

I. INTRODUCTION

Analytical (OLAP) workloads against a data warehouse rarely stay fixed: query patterns rotate between product-, customer-, and temporal-centric access paths as different business questions are asked, while an ETL pipeline continues to insert rows concurrently. A physical design (indexes, partitioning) that is tuned once for an initial workload can become sub-optimal as the access pattern shifts, yet re-tuning by hand is slow and error-prone. This project studies whether a closed-loop adaptive engine, one that watches runtime statistics and creates or drops indexes automatically under explicit safety guards, outperforms a conventional static baseline design as workload type and ETL ingestion rate vary.

Both systems are designed to run on the same PostgreSQL 16 star schema and to be evaluated across the same 3×4 experiment matrix (three workload types crossed with four ETL ingestion rates), producing twelve configurations (C1–C12) run once under each design. This report covers progress to the midterm checkpoint, at which the data layer is complete and verified against a live PostgreSQL 16 instance, and the remaining layers are specified precisely enough to be built directly against the interfaces already in place. This report makes its case in four parts: a verified data layer with a

deliberately under-provisioned baseline index set, a skewed synthetic loader validated against a live database, a fully specified 3×4 experiment matrix (Section III), and an explicit threshold-based adaptive decision rule (Section III) whose safety guards are treated as tunable experiment variables rather than fixed constants.

II. PROJECT PROGRESS BREAKDOWN

A. Task Breakdown Structure

Table I lists the project’s task breakdown, current status, and inter-task dependencies. The project is organized into four phases mirroring the system’s data flow: (1) foundation and data layer, (2) workload and ETL simulation, (3) the adaptive engine itself, and (4) experiment execution and analysis. Phase 1 is complete; Phases 2–4 are remaining and are scheduled between the midterm and final-demo checkpoints.

B. Evidence of Progress

Since project initiation, the team has moved from the approved proposal to a running, version-controlled codebase. Concretely, the repository now provisions a fully reproducible PostgreSQL 16 environment with `pg_stat_statements` preloaded, creates the five-table star schema with a partitioned fact table, installs the static baseline index set that will serve as the experimental control group, creates the three-table experiment-logging schema that both the future harness and the future adaptive engine will write into, and generates a realistically skewed synthetic dataset at the scale defined in `src/config.py` (2,000 items, 20,000 customers, 200 stores, and 500,000 fact rows spread across four yearly partitions). This constitutes a working data-layer prototype: the schema can be created, populated, and queried end-to-end today, which is the foundation every remaining phase builds on.

The load was verified against a live PostgreSQL 16 instance rather than assumed correct: the four yearly partitions hold approximately 16,000 (2022) rising to 255,000 (2025) fact rows, confirming the intended recency skew, and the single hottest item accumulated roughly 167,000 sales with a sharp Zipf drop-off, confirming the item skew the future adaptive engine will need to detect. A six-point verification script

TABLE I
TASK BREAKDOWN STRUCTURE (TBS)

ID	Task	Status	Depends on	Notes
T1	Environment setup: Docker Compose, PostgreSQL 16, pg_stat_statements preload	Completed	—	Reproducible one-command bring-up (make db-up) for all team members.
T2	Star schema design: dim_item, dim_customer, dim_store, dim_date, fact_sales	Completed	T1	fact_sales is range-partitioned by year on date_key (2022–2025 + default catch-all) to support later partition-pruning analysis.
T3	Static baseline index set (control group)	Completed	T2	Foreign-key and obvious dimension indexes only; deliberately incomplete so the adaptive engine has room to demonstrate value.
T4	Experiment logging schema (experiment_run, query_sample, adaptive_action)	Completed	T1	Structured logging table design, ready to receive rows once Phase 2–3 write into it.
T5	Synthetic data generator (Zipf-skewed items/customers, recency-skewed dates)	Completed	T2	TPC-DS-shaped schema so real dsdgen output can be substituted later without touching downstream code.
T6	Workload generator (3 query pools: product-/customer-/temporal-centric)	Remaining	T2	Query templates specified in Section III; not yet implemented in the repository (src/workload is currently an empty package placeholder).
T7	Concurrent ETL injector (zero/low/medium/high ingestion rates)	Remaining	T2, T5	Design specifies streaming incremental rows at configurable batch/interval pairs; src/etl is currently an empty placeholder.
T8	Single-run validation harness (drive one workload pool + one ETL rate, log to query_sample)	Remaining	T6, T7	src/harness is currently an empty placeholder.
T9	Adaptive engine – monitor (poll pg_stat_statements)	Remaining	T1, T4	Will sample statement statistics on a fixed interval to detect hot access paths.
T10	Adaptive engine – decision logic (seq-scan-ratio threshold, min-call filter)	Remaining	T9	Depends on the monitor’s output stream.
T11	Adaptive engine – guarded executor (CREATE/DROP INDEX with DDL timeout, index cap)	Remaining	T10	Applies runtime DDL only within safety guards to avoid lock contention with concurrent ETL.
T12	Full experiment harness (drive the 12-config × 2-mode matrix)	Remaining	T8, T11	Extends the single-run harness (T8) with an outer loop over all 12 configurations and both modes.
T13	Analysis and visualization (Matplotlib: latency, index usage, plan regressions)	Remaining	T12	Produces the static-vs-adaptive comparison charts for the final report.

(verify.sql) consolidates these checks, plus partition existence, partition-pruning behaviour, baseline index presence, and log-table presence, into a single repeatable command.

The workload generator, ETL injector, and harness (T6–T8) remain placeholders in the repository at this time; their design is given below so implementation can begin directly against the interfaces the data layer already exposes.

III. SYSTEM ARCHITECTURE AND NOVELTY

Figure 1 shows the target closed-loop architecture. A workload generator and an ETL injector would both act on the same PostgreSQL 16 instance; pg_stat_statements observes every query executed. The adaptive engine’s monitor would poll this statistics view on a fixed interval (10s by default), its decision component would flag a column as an indexing candidate once its sequential-scan ratio exceeds a 30% threshold with at least 5 calls observed, and its executor would then issue the corresponding CREATE/DROP INDEX statement under an explicit DDL timeout and a cap on total adaptive indexes, so runtime schema changes cannot stall concurrent ETL writers or grow unbounded. Every action would be written to the adaptive_action log table for

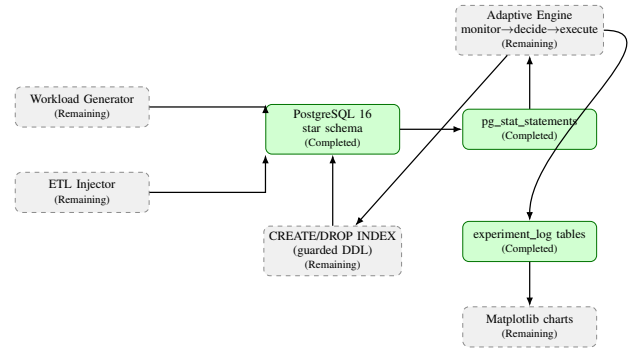


Fig. 1. Target closed-loop architecture. The monitor samples pg_stat_statements, the decision component evaluates threshold rules, and the executor applies guarded DDL back onto the same PostgreSQL instance the workload generator and ETL injector drive concurrently. Green = completed and verified; grey/dashed = designed, not yet implemented.

a full audit trail. Green components are implemented and verified; grey/dashed components are designed but not yet implemented.

The intended novelty is twofold: (1) the adaptation loop

would be driven purely by PostgreSQL’s own runtime statistics rather than an offline workload sample, reacting to distribution shift as it happens; and (2) safety guards are treated as first-class experiment variables (timeout, index cap, threshold, minimum calls), so the final analysis can quantify the adaptation-versus-overhead trade-off directly rather than reporting only a single before/after comparison. A smaller insight already realized in Phase 1: the baseline index set was deliberately under-provisioned so that any benefit the adaptive engine later demonstrates can be attributed to its own decisions rather than to a strawman baseline.

A. Query Workload Design

Three query pools are specified for T6, each isolating a different access path through the star schema so the adaptive engine’s response to a workload *shift* can be measured cleanly. The product-centric pool joins `fact_sales` to `dim_item` and filters/aggregates on category and brand, stressing the item foreign key. The customer-centric pool joins to `dim_customer` and groups by segment and region. The temporal-centric pool filters directly on `date_key` ranges and relies on existing partitioning, serving as a control for isolating partition-pruning effects from indexing effects. Each pool will be a small library of parameterized SQL templates the workload generator can draw randomized instances from at a configurable rate.

B. ETL Injection Design

The ETL injector (T7) runs alongside the workload generator, inserting batches of new `fact_sales` rows into the current partition at one of four configurable rates (zero/low/medium/high), expressed as rows per batch and seconds per interval in `src/config.py`. Running the same query pool across all four rates lets the final analysis separate latency degradation from data growth versus degradation or improvement caused by the adaptive engine’s own DDL competing with concurrent writers for locks.

C. Experiment Matrix and Metrics

The full experiment matrix crosses the three workload pools with the four ETL rates for twelve configurations: C1–C4 (product-centric at zero/low/medium/high ETL), C5–C8 (customer-centric), and C9–C12 (temporal-centric). Each configuration will be executed once under the static baseline and once under the adaptive engine (T12), for 24 total runs, with every query logged to `query_sample` and every index change logged to `adaptive_action`. The primary metrics are per-query latency (p50/p95), sequential-scan ratio before/after each adaptive action, adaptive index changes per run, and DDL wait time from ETL lock contention – the four metrics T13’s comparison charts will plot.

D. Adaptive Decision Rule and Safety Guards

The decision component’s core rule (T10) is a simple, auditable threshold test: for a column observed in `pg_stat_statements`, let r be its sequential-scan ratio

and n its call count since the last reset. A column becomes an indexing candidate when $r > 0.30$ and $n \geq 5$; an existing adaptive index becomes a drop candidate once r stays below threshold for two consecutive 10s polling intervals, avoiding thrashing on a single noisy sample. Two further guards bound the executor: a DDL timeout prevents `CREATE/DROP INDEX` from blocking behind concurrent ETL writers, and an index cap prevents unbounded growth. All four parameters are swept as independent variables in the final experiment rather than hard-coded.

IV. GITHUB REPOSITORY AND VERSION CONTROL EVIDENCE

The project is tracked on GitHub at <https://github.com/bhavyapatel0201/adaptive-warehouse>, with the Phase 1 code, SQL, and documentation under version control. The repository is organized by concern: `sql/` holds the schema, baseline-index, and logging DDL scripts in execution order, while `src/` holds one Python package per phase (workload, etl, adaptive, harness), currently empty placeholders reserving the module structure. A `Makefile` and `docker-compose.yml` give every team member an identical, one-command environment. Table II summarizes each member’s assigned and completed tasks.

TABLE II
TEAM EFFORT: ASSIGNED AND COMPLETED TASKS

Member	Assigned Task(s)	Status / Notes
Bhavya Patel	T1: Environment setup; GitHub coordination	Completed – Dockerized PostgreSQL 16, <code>pg_stat_statements</code> , <code>Makefile/.env</code> workflow, repo structure.
Yash Prajapati	T2, T3: Star schema and baseline indexes	Completed – Five-table star schema, partitioned fact table; incomplete baseline index set as control group.
Harsh Patel	T5: Synthetic data generator	Completed – TPC-DS-shaped Zipf/recency skew models, verified at target scale.
Venkata Charan Sai Doniparthi	T4: Logging schema, verification, report drafting	Completed – Logging schema, six-point <code>verify.sql</code> ; drafted this report and Phase 2 design.

V. DEMONSTRATION OF COMPLETED WORK

The following components are runnable and can be demonstrated live, directly against the artifacts already in the repository: `make db-up` brings up PostgreSQL 16 with `pg_stat_statements` preloaded; `make schema` applies `01_schema.sql`, after which `\d fact_sales` in `psql` shows the four yearly range partitions plus the default catch-all; `make baseline` applies the static control-group indexes and `make load` runs the data generator, after which

`EXPLAIN (COSTS OFF)` on a query filtered to a single year confirms only the matching yearly partition (e.g., `fact_sales_2024`) is scanned; and `verify.sql` runs all six Phase 1 checks inside the container in one command with pass/fail-readable output. Every item here corresponds one-to-one with a task marked Completed in Table I. Workload-pool queries, ETL injection, and adaptive indexing are not demonstrated in this deliverable, since T6–T11 are not yet implemented.

VI. RISKS AND NEXT STEPS

The primary risk to the remaining schedule is that the adaptive engine (T9–T11) is both the most novel and the most technically uncertain component: guarded `CREATE INDEX CONCURRENTLY` execution against a live, concurrently-written table has more failure modes than the already-verified Phase 1 work, particularly lock contention at high-ingestion-rate configurations (C4, C8, C12), which the design mitigates with the `ddl_timeout_ms` guard. A secondary risk is that TPC-DS’s official `dsdgen` tool requires a licensed download from `tpc.org`; this is mitigated by the synthetic generator already producing TPC-DS-shaped data, so real TPC-DS data can be substituted later without changing any workload, ETL, or adaptive-engine code. The team’s immediate next steps are to implement T6–T8 (workload generator, ETL injector, and single-run harness), since these are lower-risk, more mechanical tasks that are prerequisites for the adaptive engine and will also produce the project’s first real latency numbers. A tertiary risk is that `pg_stat_statements` accumulates statistics cumulatively, so the monitor must reset or delta counters between polling intervals. The team’s target schedule allocates weeks 1–2 to T6–T8, weeks 3–4 to T9–T11, and weeks 5–6 to T12–T13.

VII. CONCLUSION

At the midterm checkpoint, the project has a complete and verified data layer: a partitioned star schema, static baseline indexes, an experiment-logging schema, and a skewed synthetic data loader, all confirmed against a running database rather than assumed correct. The workload generator, ETL injector, adaptive engine, and full experiment harness remain to be implemented; their design and the interfaces they will build against are specified in Section III so that implementation can proceed directly from this point.

REFERENCES

- [1] Transaction Processing Performance Council, “TPC Benchmark DS,” <https://www.tpc.org/tpcds/>.
- [2] PostgreSQL Global Development Group, “`pg_stat_statements`,” PostgreSQL 16 Documentation, <https://www.postgresql.org/docs/16/pgstatstatements.html>.
- [3] PostgreSQL Global Development Group, “Table Partitioning,” PostgreSQL 16 Documentation, <https://www.postgresql.org/docs/16/ddl-partitioning.html>.